항상 고민하는 개발자
이경돈 입니다.

Backend Java Spring Boot

PORTFOLIO

Contact

📱010-6482-8864 | 📧 Kyeongdon.lee97@gmail.com

Profile

이 경 돈

Backend Developer

Email Kyeongdon.lee97@gmail.com

GitHub https://github.com/wgrgwg

Blog https://wgrgwg.tistory.com/

LinkedIn https://www.linkedin.com/in/kyeongdon-lee/

About me

안녕하세요! 백엔드 개발자 이경돈입니다.

- **문제 상황을 분석하고 해결 방안을 고민하는 과정**을 좋아합니다.
- 학습한 내용을 **꾸준하게 기록**하며 **지속적인 성장을 추구**합니다.
- 새로운 기술을 두려워하지 않고 **도전하며 배우는 과정**을 중요하게 생각합니다.

Education

2024.09 ~ 2025.02	**이스트소프트** 백엔드 개발 부트캠프 7기
2023.03 ~ 2025.02	**충남대학교** 컴퓨터융합학부 졸업
2021.03 ~ 2023.02	**한밭대학교** 정보통신공학과

Certificate

2026.01	OPIc IH
2025.06	정보처리기사
2023.07	SQLD

Skill

Java, Spring Boot, Spring Data JPA, MySQL, Git, Spring Security, Redis, Docker, Linux(Ubuntu), GCP Compute Engine, Caddy, GitHub Actions

Project – Somniverse

Somniverse – 꿈 기록 서비스 GitHub Site

개인 프로젝트 (2025.06~2025.09)

개요

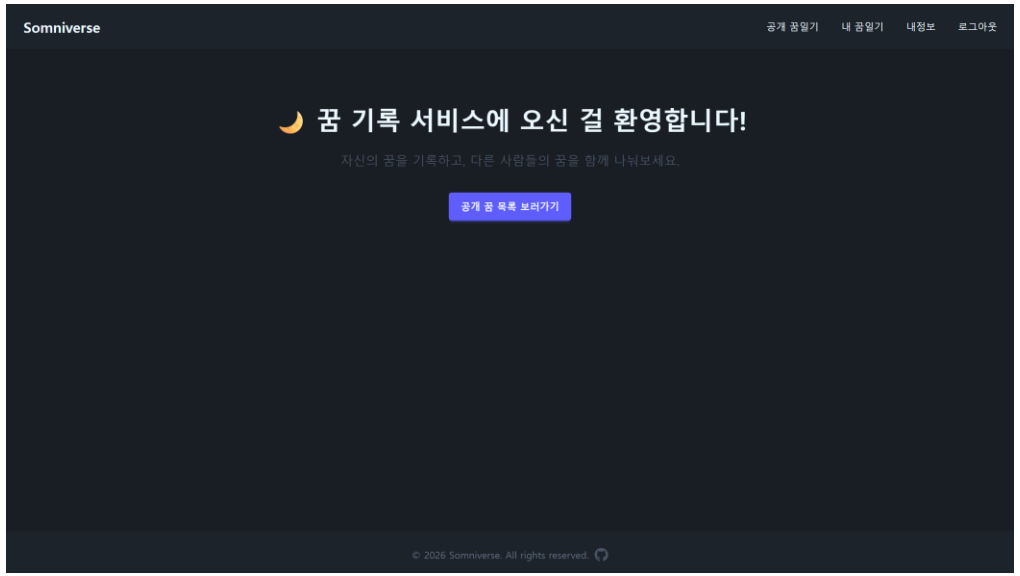
Somniverse는 사용자가 자신의 꿈을 기록하고 꿈에 담긴 감정을 분석하며 다른 사람들과 꿈을 공유하고 공감과 의견을 나눌 수 있는 꿈 기록 서비스입니다.

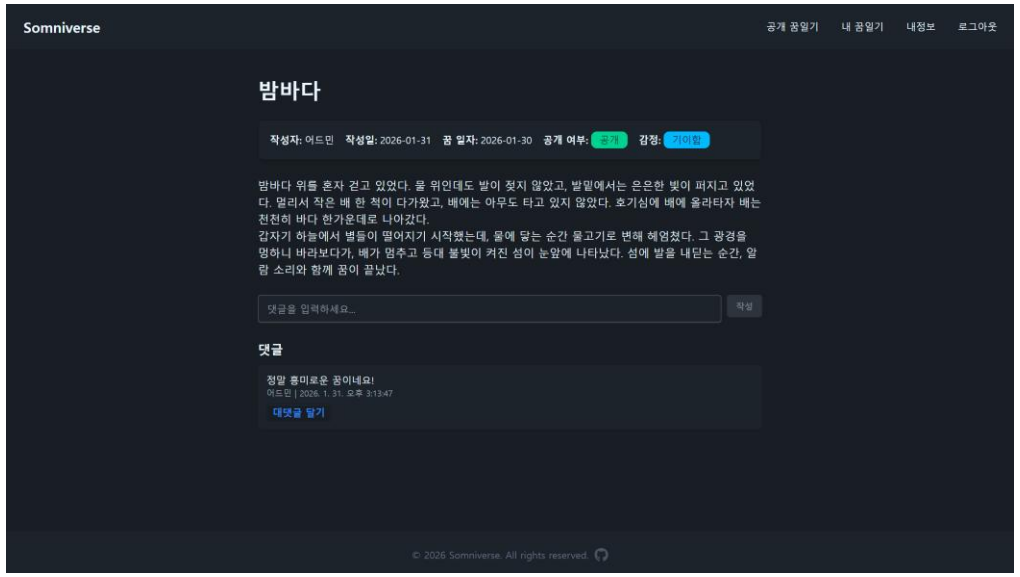
기능

- 서버 및 RESTful API 개발, 핵심 기능 구현
- ERD 기반 DB 설계
- JWT, OAuth2 인증 시스템 구현
- Gemini API 활용 감정 분석 기능 구현
- JUnit5 기반 테스트 코드 작성(Mockito, @WebMvcTest 활용)
- 배포 환경 구축 및 CI/CD 파이프라인 구성

기술

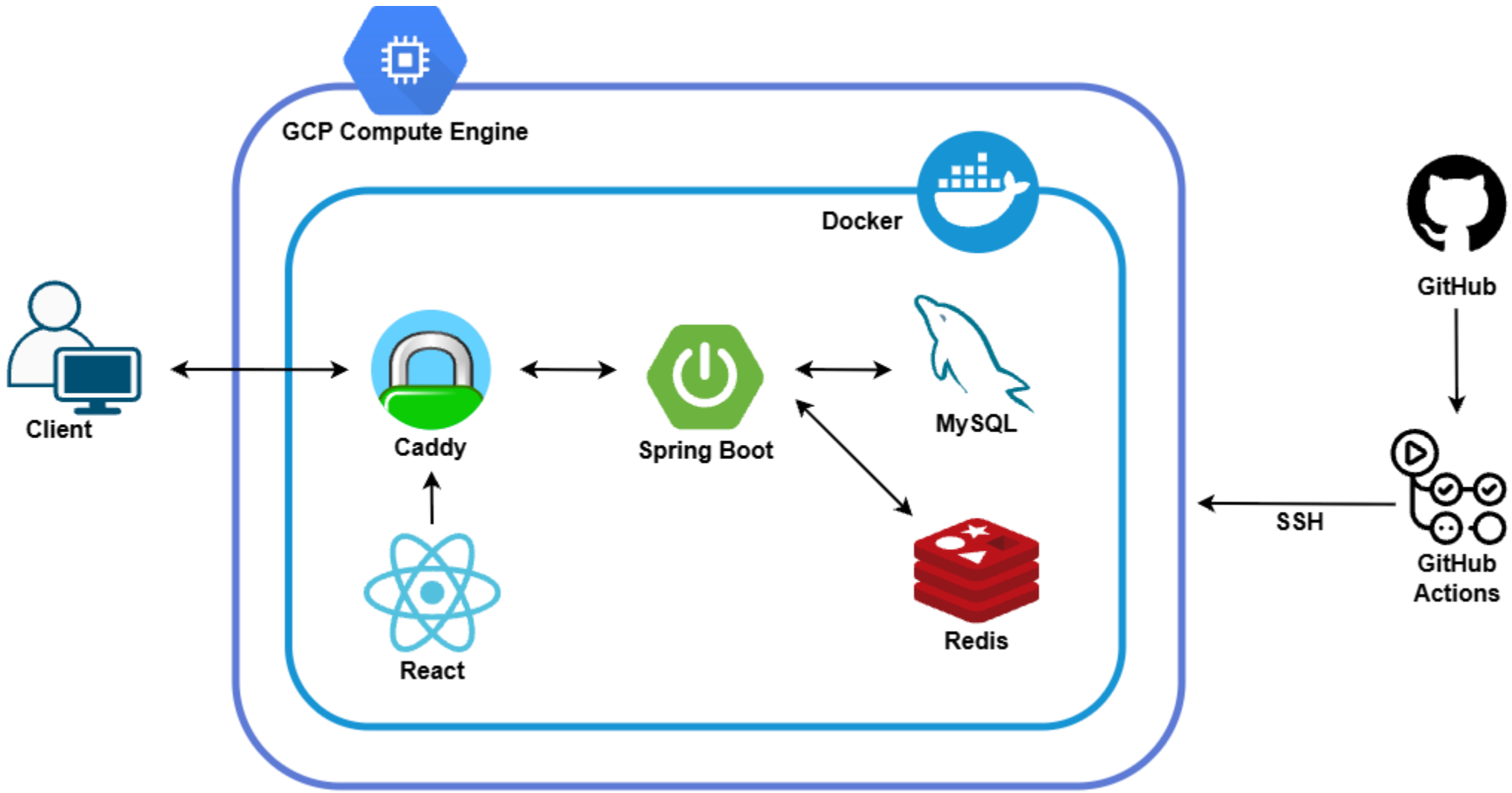
- Java, Spring Boot, Spring Data JPA, Spring Security, JUnit5
- MySQL, Redis
- Docker, GCP Compute Engine, GitHub Actions, Caddy





Project – Somniverse

Architecture



Project – Somniverse

POST 요청 재시도 시 중복 생성 방지 Blog

문제 클라이언트가 POST 요청을 재시도하는 과정에서 데이터 정합성 문제가 확인되었습니다.
POST 메서드는 기본적으로 멱등하지 않기 때문에 서버가 첫 번째 요청을 처리했음에도
동일한 요청이 다시 들어오는 경우 동일한 리소스가 중복 생성되는 현상이 발생했습니다.

탐색

- 프론트엔드 버튼 비활성화: 네트워크 레벨의 재시도는 차단 불가능, 데이터 정합성의 책임은 백엔드에 있음
- DB 유니크 제약 설정: 데이터 중복 삽입 방지는 가능, 동일한 응답 반환은 불가능
- **멱등키 패턴: 동일한 요청에 대해 서버가 같은 작업으로 식별하고 동일한 응답 반환**

요청

```
POST /api/dreams

{
  "title": "하늘을 나는 꿈",
  "content": "구름 위를 날아다녔다."
}
```

→

```
201 Created

{
  "dreamId": 101,
  "title": "하늘을 나는 꿈",
  "content": "구름 위를 날아다녔다."
}
```

응답1

재요청

```
POST /api/dreams

{
  "title": "하늘을 나는 꿈",
  "content": "구름 위를 날아다녔다."
}
```

→

```
201 Created

{
  "dreamId": 102,
  "title": "하늘을 나는 꿈",
  "content": "구름 위를 날아다녔다."
}
```

응답2

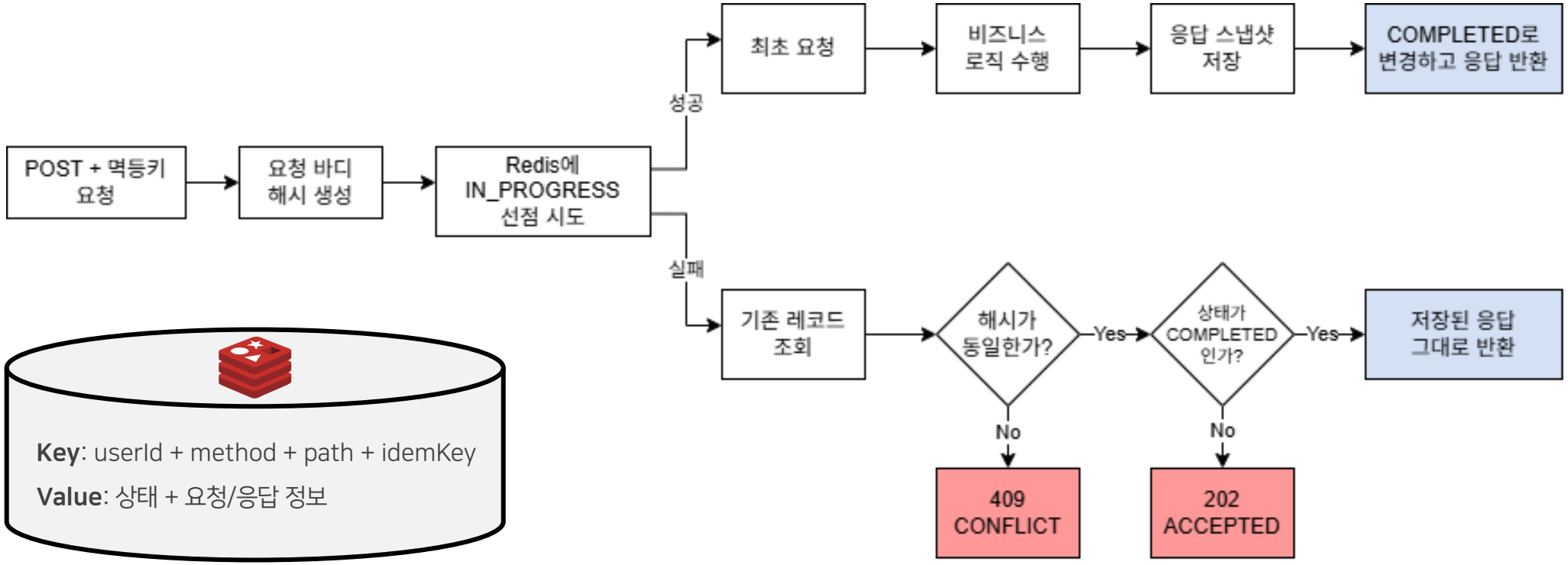
동일한 데이터 중복 생성

Project – Somniverse

POST 요청 재시도 시 중복 생성 방지 Blog

해결

- 클라이언트가 요청 헤더에 고유한 멱등키를 포함하여 보내도록 하고 Redis를 활용해 키와 요청/응답 정보를 매핑하여 저장했습니다.
- 요청 진입 시 Redis를 조회하여 동일한 멱등키 기록이 존재할 경우, **비즈니스 로직을 재실행하지 않고 저장된 최초 응답을 즉시 반환**하도록 처리했습니다.
- 키가 동일하지만 요청 내용이 다른 경우에는 409로 차단하고, 동일한 요청이 아직 처리 중인 경우에는 202를 반환하도록 설계했습니다.



Project – Somniverse

POST 요청 재시도 시 중복 생성 방지 Blog

검증 구현 이후 동일 멱등키 재시도 테스트와 k6 부하 테스트를 통해 동작을 검증했습니다.
그 결과 동일한 POST 요청이 반복되어도 게시글은 한 번만 생성되었고, 처리 중인 동일 요청에는 202 응답이 일관되게 반환되는 것을 확인했습니다.

정리 기능 자체의 구현도 중요하지만 타임아웃이나 동시 요청 같은 예외 상황을 고려하며 구현하는 것도 중요하다는 것을 배웠습니다.

IN_PROGRESS 202: 32회 발생

```
TOTAL RESULTS

checks_total.......: 29000   316.432337/s
checks_succeeded...: 100.00% 29000 out of 29000
checks_failed......: 0.00%   0 out of 29000

✓ login 200
✓ idempotent-ok

CUSTOM
created201_count...............: 28967 316.072259/s
in_progress_202................: 32    0.349167/s
```

삽입된 데이터 수: 1개

	`count(*)`
1	1

Project – Somniverse

N+1 문제 해결 Blog

문제

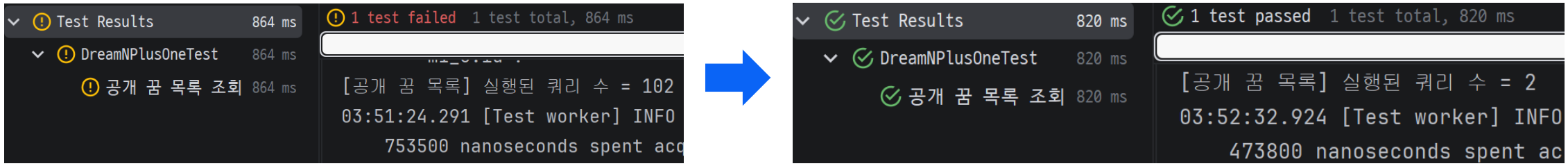
게시글 목록 조회 시 게시글과 연관된 작성자 정보를 참조할 때마다 지연 로딩 쿼리가 추가로 발생했습니다.
이로 인해 단순 목록 조회임에도 총 N+2회의 쿼리가 실행되어 응답 속도가 저하되었습니다.

탐색

- EntityGraph: 코드 및 설계가 복잡해짐
- Batch Fetching: 쿼리 감소는 가능하지만 여전히 추가 쿼리가 발생
- **Fetch Join**

해결

JPQL의 JOIN FETCH로 연관 엔티티를 한 번에 조회하는 Fetch Join 방식으로 해결하였습니다.
조회 결과가 커져 메모리 부담이 생기거나 페이징 안정성이 저하될 수 있는 한계가 있지만,
toOne 관계에서는 안전하게 최적화가 가능하기에 선택했습니다.
결과적으로 목록 조회 과정에서 발생하던 쿼리 수를 2회로 줄일 수 있었습니다. **(N+2회 → 2회)**



< Dream 조회 로직 쿼리 개수 test >

Project – Somniverse

N+1 문제 해결 Blog

문제

페이징된 댓글의 대댓글 개수를 계산하는 로직을 구현 중 지연 로딩으로 인해 추가 쿼리가 반복되는 문제를 경험했습니다.
해결을 위해 먼저 Fetch Join 방식을 고려하였으나, toMany 관계였기 때문에 부모 댓글 1건이 대댓글 수 만큼 증폭되는 문제가 있었습니다.

탐색

- Fetch Join: toMany 관계에서는 페이징 사용 불가능
- Batch Fetching: 쿼리 감소는 가능하지만 여전히 추가 쿼리가 발생
- **2단계 조회**

해결

안정적인 Paging과 성능 확보를 위해 2단계 조회 방식으로 설계를 변경하였습니다.
1차로 부모 댓글 목록을 페이지로 조회하고 2차로 현재 페이지의 부모 댓글 ID 목록을 기준으로 대댓글 수를 계산했습니다.
결과적으로 쿼리를 **3회로** 최소화하면서도 페이지 안정성을 보장했습니다.

Before: N+2회

1. 댓글 조회: **1회**
2. 전체 댓글 개수 조회(페이징): **1회**
3. 조회된 N개의 댓글마다 대댓글 조회: **N회**

→

After: 3회

1. 최상위 댓글 페이지 조회: **1회**
2. 전체 댓글 개수 조회(페이징): **1회**
3. 조회된 댓글들의 대댓글 개수 조회: **1회**

Project – Somniverse

N+1 문제 해결 Blog

정리 특정 문제에 대한 해결 방안이 여러 개 존재할 때, 각자의 특징을 고려해서 상황에 맞는 방법을 선택하여 최적화하는 것이 중요하다는 것을 배웠습니다.

방식	핵심 원리	장점	단점/주의점	권장 상황
Fetch Join	JPQL에서 JOIN FETCH를 통해 연관 엔티티 한번에 조회	단일 쿼리로 N+1 제거	toMany 관계에서 사용 불가	toOne 관계
EntityGraph	EntityGraph(attributePaths)로 fetch 전략 선언	표준적이고 재사용성 높음	• 구현체마다 조인/쿼리 방식 다름 • 연관 관계가 복잡할 경우 성능 저하	여러 엔티티 fetch 계획을 코드로 선언 시
Batch Fetching	Lazy 로딩시 여러 엔티티를 IN 쿼리로 묶어서 조회	• N+1 → N/size로 감소 • 페이징과 호환	• 여전히 1+N/size 쿼리 발생 • 튜닝 필요	toMany 관계 + Paging
2단계 조회	부모 Page 조회 → 자식 IN 조회 후 맵핑	• 쿼리 최소화 • 페이징과 호환	맵핑 로직 코드 추가 필요	toMany 관계 + Paging

Project – Somniverse

API 과부하 방지 및 서버 리소스 보호 (Rate Limit) Blog

문제 특정 IP나 사용자가 짧은 시간동안 과도한 API 요청을 보낼 경우 서버 리소스가 고갈되어 전체 서비스 장애로 이어질 위험이 있었습니다.
특히 로그인 시도나 AI API 호출과 같이 리소스 소모가 큰 로직의 경우 **과도한 요청에 대해서 서비스 안정성 문제가 발생**할 위험이 있었습니다.

탐색

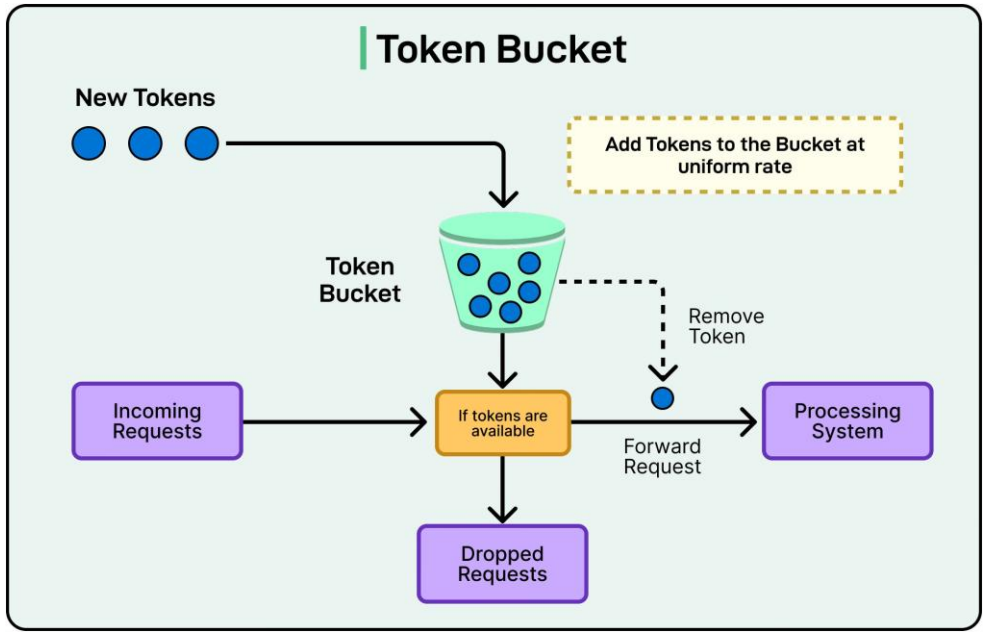
- 인프라 레벨의 요청 제한: 애플리케이션 계층에서 세부 정책 제어 불가능
- CAPTCHA 인증 도입: 요청 빈도 제어 불가능
- **Rate Limit 기법 도입(토큰 버킷 알고리즘 사용)**

토큰 버킷 알고리즘

최대 개수가 정해진 버킷에 일정한 속도로 토큰이 채워지고,
요청이 들어올 때마다 토큰을 하나씩 소비하는 제어 방식

↓

일시적인 과도한 트래픽도 어느정도 허용하면서
평균 요청 속도도 제어 가능



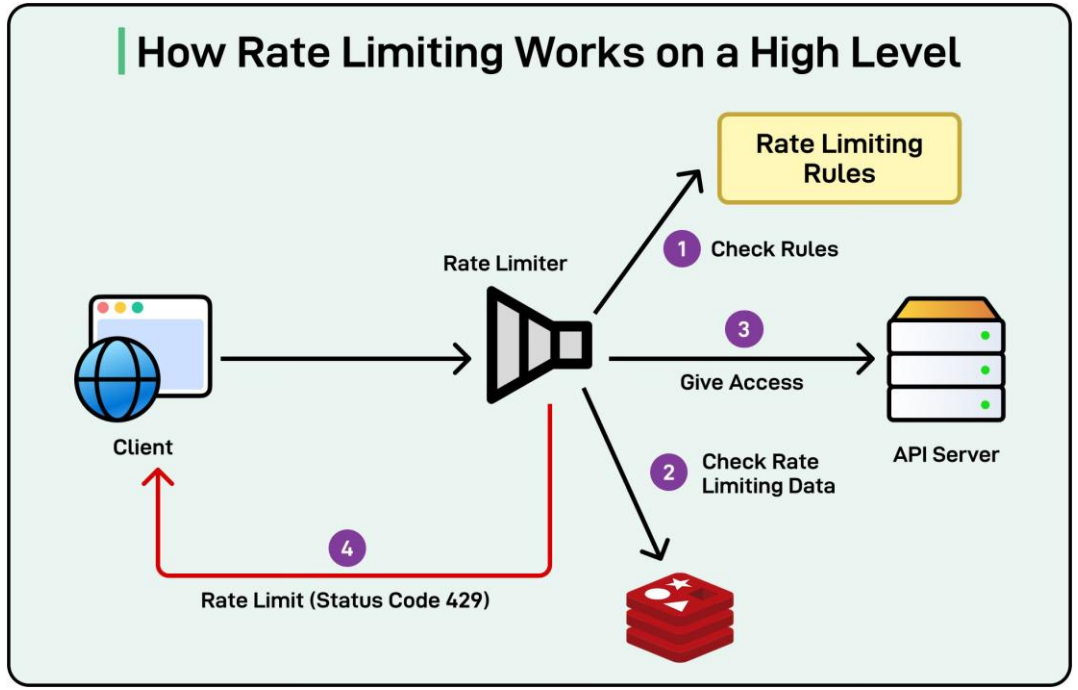
이미지 출처: https://blog.bytebytego.com/p/a-guide-to-rate-limiting-strategies

Project – Somniverse

API 과부하 방지 및 서버 리소스 보호 (Rate Limit) Blog

해결 토큰 버킷 알고리즘을 구현한 Bucket4j 라이브러리를 도입하여, 클라이언트(IP 또는 User 식별자)로 단위 시간당 요청 허용량을 설정했습니다.
결과적으로 **허용량을 초과한 요청에 대해서는 즉시 429 Too Many Requests를 반환**하여 비즈니스 로직 실행을 차단함으로써
악의적인 트래픽이나 급격한 부하로부터 서버 리소스를 보호하고 서비스 안정성을 높였습니다.

Rate Limit 동작 흐름



RateLimitFilter 토큰 소비 실패, 성공

```java
// 1. 버킷 조회 및 토큰 소비
String bucketKey = computeBucketKey(request, policy.keyStrategy(), policy.name());
BucketProxy bucket = proxyManager.getProxy(bucketKey, policy::bucketConfiguration);
ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(TOKENS_PER_REQUEST);

// 토큰 소비 실패 시 429 반환
if (!probe.isConsumed()) {
    long seconds = TimeUnit.NANOSECONDS.toSeconds(probe.getNanosToWaitForRefill());
    rateLimitResponseWriter.writeRetryAfter(
        response,
        RateLimitErrorCode.TOO_MANY_REQUESTS,
        seconds,
        NO_REMAINING_TOKENS,
        policy.name(),
        registry.isAddHeaders()
    );
    return;
}

// 토큰 소비 성공 시 정상 흐름
rateLimitResponseWriter.writeSuccessHeaders(
    response,
    probe.getRemainingTokens(),
    registry.isAddHeaders()
);
filterChain.doFilter(request, response);
```

이미지 출처: https://blog.bytebytego.com/p/a-guide-to-rate-limiting-strategies

Project – Somniverse

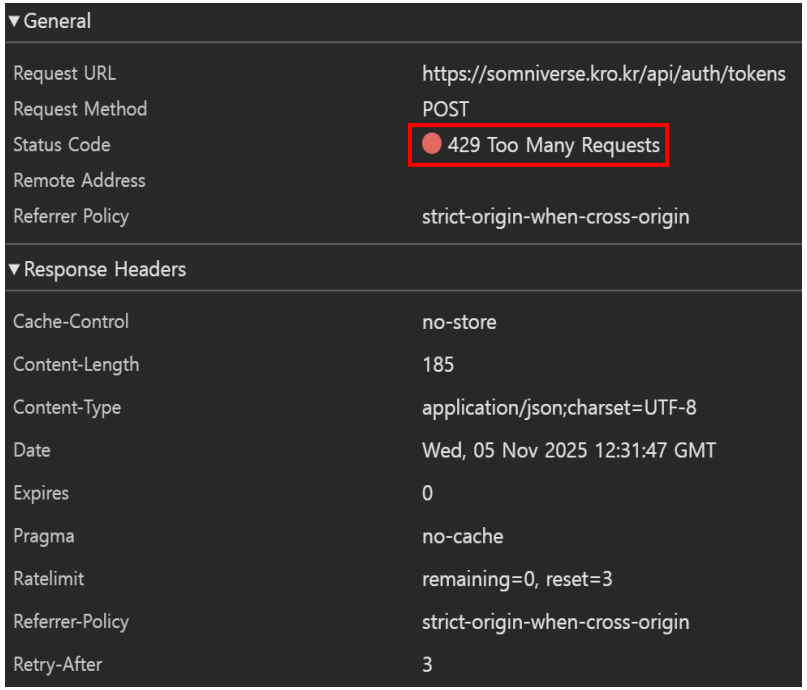
API 과부하 방지 및 서버 리소스 보호 (Rate Limit) Blog

검증 구현 이후 실제 API 호출을 통해 임계치를 초과한 요청이 비즈니스 로직에 진입하기 전에 정상적으로 429 응답으로 차단되는 것을 확인했습니다.

정리 Rate Limit으로 요청을 강하게 막는다면 과도한 차단으로 사용자 경험이 저하될 수 있습니다.
이 경험을 통해서 안정성과 사용성 사이의 균형을 맞추는 설계가 중요하다는 것을 배웠습니다.

로그인 API에 적용

429 응답 확인



▼General

Request URL	https://somniverse.kro.kr/api/auth/tokens
Request Method	POST
Status Code	429 Too Many Requests
Remote Address	
Referrer Policy	strict-origin-when-cross-origin

▼Response Headers

Cache-Control	no-store
Content-Length	185
Content-Type	application/json;charset=UTF-8
Date	Wed, 05 Nov 2025 12:31:47 GMT
Expires	0
Pragma	no-cache
Ratelimit	remaining=0, reset=3
Referrer-Policy	strict-origin-when-cross-origin
Retry-After	3

Project – WordWise

WordWise – 영어 단어 학습 서비스

GitHub

팀 프로젝트 - 기여도 40% (2025.01~2025.02)

개요

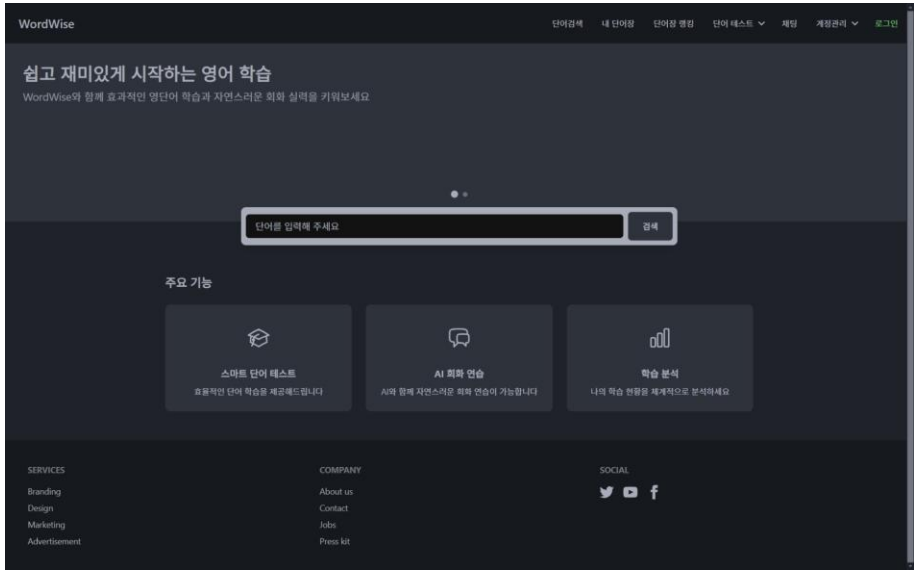
WordWise는 사용자가 단어의 뜻만 외우는 것이 아닌 예문을 생성해서 공부함으로써 단어의 여러 사용법을 익힐 수 있는 영어 단어 학습 서비스 입니다.

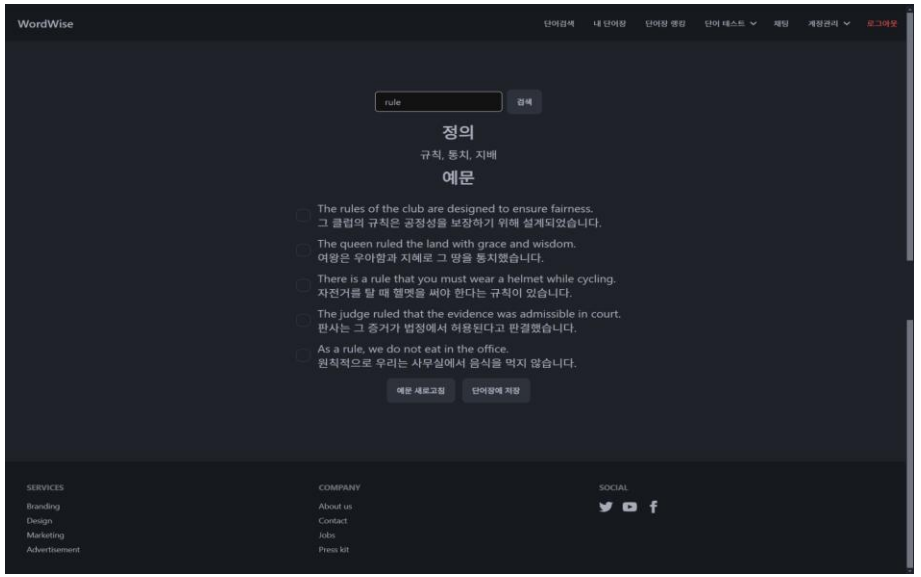
역할

- 서버 및 RESTful API 개발
- ERD 기반 DB 설계
- 생성형 AI API 활용 영어 단어 예문 생성 기능 구현

기술

- Java, Spring Boot, Spring Data JPA
- MariaDB
- Git 브랜치 전략에 따른 협업 개발





Project – WordWise

연관관계 편의 메서드 추가

문제 양방향 연관관계로 설계된 Word와 Example 엔티티에서 Example.from 메서드로 예문 생성 시 Example의 word 값만 세팅하고, Word의 examples 컬렉션은 세팅하지 않는 문제가 있었습니다.

Word(단어) 클래스

```java
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Word {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "word_id")
    private Long id;

    @OneToMany(mappedBy = "word", fetch = FetchType.LAZY)
    List<Example> examples = new ArrayList<>();

    // 생략
}
```

Example(예문) 클래스

```java
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Example {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "example_id")
    private Long id;

    @ManyToOne
    @JoinColumn(name = "word_id")
    private Word word;

    private String sentence;
    private String sentenceMeaning;

    private boolean deleted;

    public static Example from(Word word, ExampleDto exampleDto) {
        Example example = new Example();
        example.word = word;
        example.sentence = exampleDto.getSentence();
        example.sentenceMeaning = exampleDto.getSentenceMeaning();
        return example;
    }
}
```

Project – WordWise

연관관계 편의 메서드 추가

해결 Example과 Word는 서로 양방향 관계이고, 외래키를 갖고 있는 Example의 연관관계의 주인입니다.
따라서 주인인 Example 쪽에 word 필드를 설정하면 DB 반영은 가능하지만, 반영 전까지 Word 객체의 examples 필드는 변경되지 않습니다.

이를 해결하기 위해 연관관계 편의 메서드를 추가해 연관관계의 주인 쪽 필드 설정과 반대편 컬렉션 동기화를
하나의 메서드로 묶어 객체 상태가 일관되게 관리되도록 했습니다.

Before

```java
public static Example from(Word word, ExampleDto exampleDto) {
    Example example = new Example();
    example.word = word;
    example.sentence = exampleDto.getSentence();
    example.sentenceMeaning = exampleDto.getSentenceMeaning();
    return example;
}
```

After

```java
public static Example from(Word word, ExampleDto exampleDto) {
    Example example = new Example();
    example.setWord(word);
    example.sentence = exampleDto.getSentence();
    example.sentenceMeaning = exampleDto.getSentenceMeaning();
    return example;
}

private void setWord(Word word) {
    this.word = word;
    word.getExamples().add(this);
}
```

Project – WordWise

연관관계 편의 메서드 추가

검증 수정 이후에는 테스트를 작성해서 편의 메서드가 실제로 양쪽 참조를 동시에 적용했는지를 확인했습니다.

정리 이 경험을 통해 애플리케이션 내부에서 객체가 어떻게 관리되고 유지되고 있는지를 고려하는 습관의 필요성을 느꼈습니다.

```java
@Test
@DisplayName("Example 생성 시 Example과 Word 연관관계 동기화 확인")
void createExample_syncsWordAndExamples() {
    // given
    Word word = Word.from(WordCreateDto.of("apple", "사과"));
    ExampleDto exampleDto = ExampleDto.of("I eat an apple.", "나는 사과를 먹는다.");

    // when
    Example example = Example.from(word, exampleDto);

    // then
    assertThat(example.getWord()).isEqualTo(word);
    assertThat(word.getExamples()).contains(example);
}
```

감사합니다

Contact

010-6482-8864 Kyeongdon.lee97@gmail.com